From Zero to Production

Building a Multi-Node AI Agent with RAG, Guardrails, and Cloud Deployment

Chandra Prakash

github.com/cprakash0105/research-agent

The Problem

Most AI demos stop at "call an LLM and print the response"

- ✗ No structured reasoning or planning
- ✗ No grounding in real sources (hallucination)
- ✗ No security or governance
- ✗ No production deployment

Goal: Build something that thinks in steps, uses tools, retrieves grounded information, and does it all safely.

The Solution: Research Agent

An agentic AI assistant that autonomously:

🧠 Plans — Breaks query into targeted sub-questions
🔍 Researches — Searches web + indexes uploaded documents
📊 Analyzes — RAG retrieval for grounded synthesis
✍️ Writes — Structured report with citations
💬 Answers — Follow-up Q&A via RAG

Agent Architecture (LangGraph State Machine)

4-Node Pipeline:

User Query

↓

🧠 PLANNER → Breaks into 3-5 sub-questions

↓

🔍 RESEARCHER → Tavily search + FAISS indexing

↓

📊 ANALYST → RAG retrieval (top-k) + synthesis

↓

✍️ WRITER → Markdown report with citations

↓

💬 FOLLOW-UP Q&A → RAG against same index

RAG Pipeline

Sources (Web + Uploaded Docs)

↓ PII Redaction (Microsoft Presidio)
↓ Text Splitting (500 chars, 50 overlap)
↓ Embedding (Gemini Embedding 001, 3072 dim)
↓ FAISS Vector Index (in-memory)
↓ Similarity Search (top-k per query)
↓ Deduplication across sub-questions
↓ Context → LLM → Analysis / Answer

Key: 8 relevant chunks > 50 raw search results

Governance & Security (10 Features)

Input Protection

- 🔐 Authentication (password)
- 🚦 Rate Limiting (10 req/min)
- 🛡️ Input Validation (regex)
- 🧠 Prompt Injection Guard (LLM)
- 🔐 Encrypted Secrets (Fernet)

Output & Data Protection

- 🚫 Output Safety Filter (LLM)
- 🔐 PII Redaction (Presidio)
- 📋 Audit Trail (JSONL)
- 💰 Token Budget Tracking
- ⏰ Session TTL (30 min)

Technology Stack

AI & Backend

LangGraph (agent orchestration)
Google Gemini 2.0 Flash (LLM)
Gemini Embedding 001
Tavily (web search)
FAISS (vector store)
Presidio + spaCy (PII)
Chainlit (UI)

Infrastructure

GCP Cloud Run (serverless)
GCP Secret Manager
GCP Cloud Build (CI/CD)
Artifact Registry
Docker (python:3.11-slim)
Kind (local K8s dev)
pytest (76 automated tests)

Cloud Deployment (GCP Cloud Run)

CI/CD Pipeline:

git push → Cloud Build → Artifact Registry → Cloud Run

- Scales to zero when idle (~$0-5/month)
- Zero-downtime deployments
- Secrets from GCP Secret Manager
- Session affinity for WebSocket support
- 1Gi memory, 300s timeout, 0-2 instances

Key Takeaways

1. Agentic AI > prompt chaining
State machines with error handling per node

2. Governance isn't optional
Rate limiting, PII detection, audit trails = table stakes

3. RAG quality > RAG quantity
8 relevant chunks beat 50 raw search results

4. Middleware pattern works for AI governance
Each feature is a layer, toggled independently

5. Cloud Run = perfect for AI side projects
Scales to zero, WebSocket support, $0 when idle

Free Learning Resources

Courses:

- Building AI Agents with LangGraph — DeepLearning.AI
- AI Agentic Design Patterns — DeepLearning.AI
- Building RAG Agents with LLMs — NVIDIA DLI
- Generative AI Fundamentals — IBM CognitiveClass.ai
- Building GenAI Apps with Python — IBM CognitiveClass.ai
- Functions, Tools & Agents — DeepLearning.AI

Docs:

- LangGraph — langchain-ai.github.io/langgraph
- Gemini Cookbook — github.com/google-gemini/cookbook

Thank You

github.com/cprakash0105/research-agent
research-agent-489654189917.us-central1.run.app

Chandra Prakash